SHRI RAMSWAROOP MEMORIAL UNIVERSITY

Innovative assesement (2026-27)

TECHNICAL REPORT

ECG Signal Filtering and Analysis Using Python

SUBMITTED BY:

NAME: VIRAJ YADAV

ROLL ON: 202410101150230

COURSE:B.TECH CSE(DS+AI)

GROUP NO : 57,58

SUBMITTED TO:

Dr.Sandeep dixit

Abstract

This project demonstrates digital signal processing through an educational simulation of an electrocardiogram (ECG)-like signal. A synthetic waveform is constructed from Gaussian pulses representing approximate P, Q, R, S, and T components. The signal is contaminated with simulated baseline wander, random high-frequency noise, and 50 Hz power-line interference. A fourth-order Butterworth band-pass filter with cutoff frequencies of 0.5 Hz and 40 Hz is designed using SciPy and applied with forward-backward filtering. Time-domain plots, frequency spectra, approximate R-peak detection, and signal-to-noise ratio (SNR) calculations are used to examine the filtering process. Because the clean reference is known in this simulation, SNR can be compared before and after filtering. The project is for learning Signals and Systems concepts and is not a clinical ECG analysis tool.

1. Introduction

An electrocardiogram records electrical activity associated with the heart. ECG signals can be affected by slow baseline movement, muscle activity, and electrical power-line interference. Digital signal processing can reduce some unwanted components while retaining useful waveform features. This project uses a synthetic ECG-like signal so both the clean reference and noisy version are available. The waveform is simplified and does not represent the full variability of a real physiological ECG.

2. Objectives

- Generate a synthetic ECG-like waveform using Gaussian pulses.
- Sample the signal at 250 samples per second for 10 seconds.
- Simulate baseline wander, random noise, and 50 Hz interference.
- Design and apply a Butterworth band-pass filter from 0.5 Hz to 40 Hz.
- Compare clean, noisy, and filtered waveforms in the time domain.
- Compare noisy and filtered frequency spectra using the FFT.
- Detect approximate R peaks and calculate SNR before and after filtering.

3. Tools and Requirements

Tool / library	Purpose
Python 3	Implementation language
NumPy	Numerical arrays, time axis, random noise, and FFT
SciPy	Butterworth filter design, zero-phase filtering, and peak detection
Matplotlib	Time-domain and frequency-domain plots

Installation command:

pip install numpy scipy matplotlib

4. Methodology

4.1 Signal generation

The sampling frequency is fs = 250 Hz and duration is 10 seconds. The time vector has spacing 1/fs seconds. A synthetic heartbeat is repeated approximately once per second, corresponding to an illustrative rate of about 60 beats per minute. Each beat is approximated by Gaussian pulses representing P, Q, R, S, and T components. A

Gaussian pulse can be written as $A \cdot \exp(-((t - \mu)/\sigma)^2)$, where A is amplitude, μ is the pulse centre, and σ controls width. The selected parameters are for visualization, not medical interpretation.

4.2 Noise simulation

Three noise components are added: baseline wander represented by a 0.3 Hz sinusoid of amplitude 0.20; random noise generated from a zero-mean normal distribution with scale 0.08; and 50 Hz power-line interference of amplitude 0.04. The noisy signal is the sum of the clean waveform and these components.

4.3 Filter design

A fourth-order Butterworth band-pass filter is designed with cutoff frequencies of 0.5 Hz and 40 Hz. The Butterworth response provides a smooth passband. SciPy's second-order-section (SOS) representation is used for numerical robustness. The sosfiltfilt function filters forward and backward, providing zero-phase offline filtering. It uses the complete signal and is not intended for real-time filtering.

4.4 Frequency-domain analysis

The real-valued FFT is calculated for noisy and filtered signals. Their magnitude spectra are plotted against frequency bins to compare attenuation outside the filter passband. The displayed magnitude is not normalized to a calibrated physical amplitude spectrum.

4.5 R-peak detection

The find_peaks function is applied to the filtered signal with a minimum peak distance of 0.4 seconds and prominence threshold of 0.4. These settings are chosen for this synthetic example. Performance depends on waveform shape, noise, sampling rate, and parameter settings; the count is not a clinical heart-rate measurement.

4.6 Signal-to-noise ratio

For clean reference x[n] and test signal y[n], error is $e[n] = x[n] - y[n]$. The code estimates SNR as $10 \cdot \log10(\text{mean}(x[n]^2) / \text{mean}(e[n]^2))$ dB, first for the noisy signal and then for the filtered signal. An increase indicates lower mean-square error relative to the clean synthetic reference. Filtering can also alter desired waveform components, so SNR should be considered with visual inspection.

5. System Workflow

1. Create the time vector and synthetic ECG-like waveform.
2. Add baseline wander, random noise, and 50 Hz interference.
3. Design the Butterworth band-pass filter.
4. Filter the noisy waveform using SOS forward-backward filtering.
5. Calculate SNR before and after filtering.
6. Detect approximate R peaks in the filtered signal.
7. Plot the time-domain waveforms and compare FFT magnitude spectra.

6. Key Parameters

Parameter	Value	Meaning
Sampling frequency	250 Hz	250 samples per second
Duration	10 s	Simulation length
Baseline wander	0.3 Hz; amplitude 0.20	Slow drift
Random noise scale	0.08	Simulated broadband noise
Power-line interference	50 Hz; amplitude 0.04	Mains-frequency component
Filter	Fourth-order Butterworth	Band-pass filter
Cutoff frequencies	0.5–40 Hz	Approximate retained band

Minimum peak distance	0.4 s	Limits closely spaced detections
Peak prominence	0.4	Rejects less prominent peaks

7. Results and Discussion

When executed, the script displays three time-domain plots: the clean synthetic ECG-like signal, the noisy signal, and the filtered signal with detected R peaks marked. A second figure compares noisy and filtered FFT magnitude spectra. The terminal reports sampling frequency, duration, SNR before and after filtering, and the detected peak count.

Expected qualitative result: the noisy waveform contains slow baseline movement and rapid fluctuations. The band-pass filter should reduce much of the low-frequency baseline drift and high-frequency content above the upper cutoff. The 50 Hz interference lies above the 40 Hz cutoff and should be attenuated. The filtered waveform should appear cleaner, although filtering can also alter some desired ECG-like components.

Exact SNR values and detected peak count depend on execution. They are not reported here as measured results because the script has not been run while preparing this report. Record the values printed by your run in the final submission.

8. Advantages

- Demonstrates several core Signals and Systems concepts in one simulation.
- Provides a known clean reference for a controlled filtering experiment.
- Uses common open-source Python libraries.
- Produces visual results that can be explained in a presentation.
- Allows noise levels and filter parameters to be changed for further experiments.

9. Limitations

- The signal is synthetic and does not reproduce the full variability of clinical ECGs.
- The noise model is simplified and may not reflect real acquisition artefacts.
- A fixed 0.5–40 Hz band-pass filter is not ideal for every ECG application.
- Forward-backward filtering is suitable for offline analysis, not low-latency real-time monitoring.
- R-peak settings are demonstration parameters and are not clinically validated.
- SNR calculation requires a clean reference, which is generally unavailable for real patient recordings.

10. Future Scope

- Load real ECG data from a CSV file or documented public dataset.
- Add a notch filter around 50 Hz or 60 Hz and compare it with the band-pass filter.
- Compare Butterworth, Chebyshev, and FIR filter designs.
- Build a Streamlit interface for uploading data and selecting filter parameters.
- Measure RMSE and correlation against a clean reference in controlled simulations.
- Evaluate peak detection against labelled annotations before assessing performance.

11. Conclusion

The ECG Signal Filtering and Analysis project demonstrates sampling, noise modelling, Butterworth band-pass filtering, FFT-based spectral analysis, peak detection, and SNR calculation. It illustrates how filtering can

improve the appearance and measured quality of a known synthetic signal. Results are educational only; validation with real, appropriately labelled ECG data would be required for any clinical application.

Appendix A: Run Instructions

8. Install Python 3.
9. Install dependencies: pip install numpy scipy matplotlib
10. Save the code as ecg_analysis.py.
11. Run: python ecg_analysis.py
12. Record the SNR values and R-peak count printed by the program.
13. Insert screenshots of generated plots into the report if required.